Cropping the whitened data to a window centred on the glitch

SAGE, `eucaif` branch: `generate_data` in `virgo_data_prep.ipynb`

29 October 2026

Notation

Symbol	Meaning	In the code
t_d	GPS time of the first decimated sample	`data.t0.value`
f_s	sample rate after decimation	`fs` (4096 Hz)
N	number of decimated samples	`x_sage.numel()`
τ_f	length of the whitening filter	`fduration` (2 s)
p	samples dropped by the whitening at each end, $p = f_s\tau_f/2$	`whiten.pad` (4096)
t_0	GPS time of the glitch	`t0`
K	length of the window kept	`keep` (32 s)

Derivation

Whitened series. `FIRWhitening` convolves with a filter τ_f long, so the first and last $\tau_f/2$ of its output are corrupted and dropped: it returns $N - 2p$ samples, the first at

$$t_w = t_d + \frac{p}{f_s}, \qquad w[n] \text{ at time } t_w + \frac{n}{f_s}, \quad n = 0, \ldots, N - 2p - 1. \tag{1}$$

Sample of the glitch (the nearest one):

$$n_0 = \mathrm{round}[(t_0 - t_w)\, f_s]\,. \tag{2}$$

Start of the window: half the window before the glitch,

$$i_0 = n_0 - \frac{K f_s}{2}, \qquad \text{kept: } w[i_0], \ldots, w[i_0 + K f_s - 1] \quad (K f_s \text{ samples}). \tag{3}$$

In one line, with $t_w = t_d + \tau_f/2$ (as $p/f_s = \tau_f/2$) and $K f_s/2$ a whole number,

$$i_0 = \mathrm{round}\left[\left(t_0 - t_d - \frac{\tau_f}{2} - \frac{K}{2}\right) f_s\right]. \tag{4}$$

The rounding is needed: $t_0 - t_d$ is generally not a whole number of samples (GPS times of glitches have fractional seconds; frames start on sample boundaries).

Validity: the window must lie inside the whitened series,

$$0 \le i_0 \qquad \text{and} \qquad i_0 + K f_s \le N - 2p, \tag{5}$$

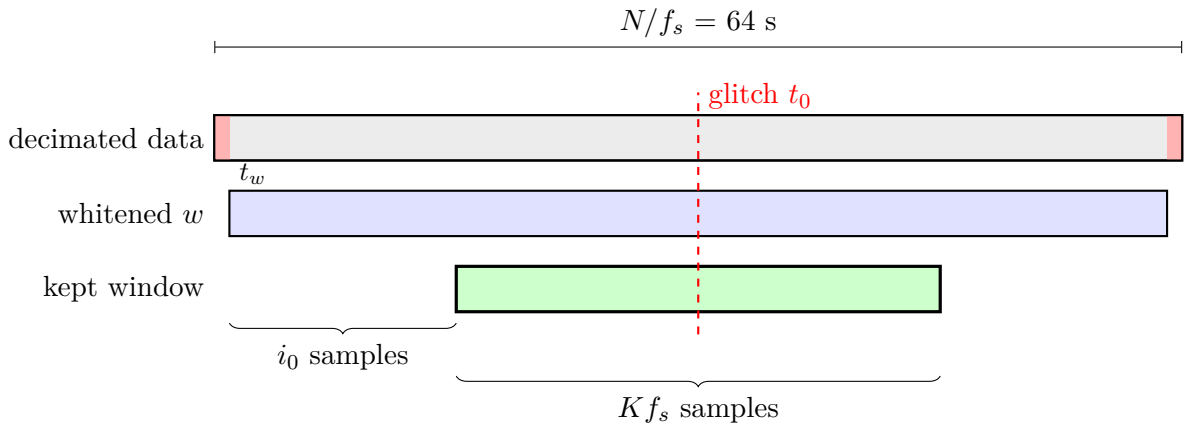
otherwise the code stops (fetch more data, or keep less).

Time of the glitch in the window (`tc` in the notebook):

$$\tau_c = t_0 - \left(t_w + \frac{i_0}{f_s}\right), \qquad \left|\tau_c - \frac{K}{2}\right| \le \frac{1}{2f_s}, \tag{6}$$

so the glitch is at the centre to within half a sample (0.12 ms at 4096 Hz). Because n_0 uses the data's actual start t_d and not the requested start, the glitch stays centred even when the frames begin a fraction of a sample off the requested time.

Picture



Red: the p samples at each end that the whitening corrupts and drops.

Example

With $f_s = 4096$ Hz, $\tau_f = 2$ s ($p = 4096$), $K = 32$ s and data starting exactly at $t_0 - 32$ s:

$$t_0 - t_w = 31 \text{ s}, \qquad n_0 = 31 \times 4096 = 126\,976, \qquad i_0 = 126\,976 - 16 \times 4096 = 61\,440,$$

which equals the earlier fixed crop, $16 \times 4096 - 4096 = 61\,440$ samples.

In the code

```
t_white0 = data.t0.value + whiten.pad / fs                          # t_w
i0 = int(round((t0 - t_white0) * fs)) - keep * fs // 2              # n_0 - K f_s / 2
if i0 < 0 or i0 + keep * fs > white_sage.numel():                   # validity
    raise ValueError("the glitch is too close to the edge of the data")
white_sage = white_sage[i0:i0 + keep * fs]                          # K f_s samples
tc = t0 - (t_white0 + i0 / fs)                                      # tau_c, ~ K / 2
```